

Refactor 1: Moving Atlas from Monolithic DAO to Repository Pattern

this document is my post-mortem on the `atlas` module refactor. I moved from a single `MaiaDAO` class that owned every database operation, to a set of small `Repository` classes that each own one entity. I'll explain why I did it, what the patterns mean, and what the end state looks like.

disclaimer: Deepseek v4 Flash (provided by opencode zen/free) was used to assist with the mechanical parts of this refactor, the actual editing of 40+ files.

What is Pleiades?

Initially named: Youtube-ML-Pipeline, it was a semester project with some ridiculous requirements. I was a student, and the project was a solo effort. I had to learn async Python, Prefect, Postgres, SQLAlchemy, and HuggingFace in 3 weeks. A live data ingestion -> Training -> Inference -> Visualization pipeline was the goal.

this repository is quite old now. As time was short at the time, and I was unfamiliar with the technologies listed, not to mention it was a solo project; I "vibe-coded" the project. Then I kept adding to the technical debt, when a new model came out with a free tier, I'd throw it at "Pleades", hoping it was good enough. And finally, I have now grown enough as a developer, to sit down, and design a maintainable refactor.

this isn't to say no thought went into the architecture. But, at the time, I wasn't a good software designer. I knew basic patterns, singleton, dao, and made a model blindly implement what looked good from a deep learning perspective. All I cared about was that the right kind of data, was being stored in a way somewhat familiar to me.

Why Pleiades?

The naming scheme was so that the project would be cool and memorable. I wanted to make 7 modules. One named after each of the 7 stars in the Pleiades star cluster. This would make the codebase easier to navigate, since I could associate each module with a concept, a greek goddess and her story.

What is Atlas?

Atlas isn't even a pleiades. But he's their dad! And so, in the repository, he is the father of the modules, holding up the database access, just like he holds up the sky in greek mythology. The `atlas` module is the database access layer for the entire pipeline. It contains adapters, DAOs, and now repositories.

Before: What Atlas Used To Be

the old `atlas` module centered around `MaiaDAO`. this was a single class, about 400 lines, that had methods for every database operation in the entire pipeline:

```
class MaiaDAO:
    async def ingest_video_metadata(self, data): ...
    async def fetch_tracker_targets(self, batch_size): ...
    async def log_video_stats_batch(self, stats): ...
    async def get_channel_by_id(self, channel_id): ...
    async def add_to_search_queue(self, term): ...
```

```
async def archive_cold_stats(self, days): ...
# ... 30+ more methods
```

every agent: hunter, tracker, scribe, painter, janitor, archeologist, imported `MaiaDAO` and used it for everything. Here is what the tracker agent used to look like at the call site:

```
from atlas.adapters.maia import MaiaDAO

dao = MaiaDAO()
targets = await dao.fetch_tracker_targets(batch_size)
```

this looks innocent enough. the problems were elsewhere.

problem 1: vault switching was fragile

the DAO had methods like `archive_cold_stats` that touched both Postgres and the vault (HF or GCP). the switching logic was:

```
def archive_cold_stats(self, ...):
    stats = self._fetch_all(...)
    v = get_vault() # global singleton, provider chosen at import time
    v.append_metrics(stats, date=...)
```

`get_vault()` returned a HuggingFace vault or a Google Cloud vault based on an environment variable at startup. swapping meant restarting the process. you couldn't have both. you couldn't test one path without patching the global.

this was not a "database abstraction." it was a monolith that happened to talk to two databases.

problem 2: tests were a patch-matching minefield

because `MaiaDAO` was a single class used everywhere, unit tests had to patch it at the module level:

```
@patch("maia.hunter.flow.MaiaDAO")
@patch("maia.tracker.flow.MaiaDAO")
```

if a consumer file renamed its import or changed which method it called, the test broke silently. integration tests were worse — they imported `MaiaDAO` directly and every fixture created a "real" instance, which meant spinning up Postgres even when testing a single method.

Table 1 — How tests touched the DAO before the refactor

Test file	Approach	Pain level
<code>maia/tests/test_hunter.py</code>	<code>@patch("maia.hunter.flow.MaiaDAO")</code>	medium — patch

Test file	Approach	Pain level
		paths fragile
<code>alkyone/tests/test_integration.py</code>	<code>@patch("maia.hunter.flow.MaiaDAO")</code>	medium — same problem
<code>alkyone/tests/test_janitor_archival.py</code>	real <code>MaiaDAO()</code> + real Postgres	high — slow, needs infra
<code>alkyone/tests/test_adaptive_scheduling.py</code>	real <code>MaiaDAO()</code> + real Postgres	high — slow, needs infra

problem 3: no domain boundaries

`MaiaDAO.ingest_video_metadata` also upserted channels. `MaiaDAO.archive_cold_stats` also wrote to vault. `MaiaDAO.fetch_tracker_targets` returned raw tuples that the caller had to know the column order of.

this made it impossible to reason about where data lived. the DAO was a god object — it knew everything about the database schema, and every consumer knew everything about the DAO.

DAO vs Repository: What These Patterns Mean

I want to be precise about these two patterns because they were quite confusing to me at first. A good resource is [cosmic python's chapter](#)

DAO (Data Access Object)

a DAO wraps raw database access. it talks in SQL, cursors, and connection pools. its methods look like this:

```
class VideoDao:
    async def insert_row(self, values: tuple) -> None: ...
    async def select_by_id(self, id: str) -> Optional[Dict[str, Any]]: ...
```

the return type is a dict or a tuple. the caller knows it's talking to a database. this is fine for a thin wrapper, but it leaks storage details everywhere.

Table 2 — DAO characteristics

Property	What it means
Language	SQL, rows, tuples, dicts
Boundary	One class per connection or table group
Return type	Raw dicts / tuples

Property	What it means
Testability	Patch the connection or spin up a real DB
Swap backend	Rewrite the SQL

Repository

a Repository sits on top of the DAO (or adapter) and speaks the language of the problem. it returns domain objects:

```
class VideoRepository:
    async def fetch_tracker_targets(self, batch_size: int) -> List[Video]: ...
    async def get_by_id(self, video_id: str) -> Optional[Video]: ...
```

the caller gets a `Video` model with `.title`, `.status`, `.model_dump()`. it does not know about SQL. it does not know about connection pools. it does not care.

here is the critical distinction in one sentence:

a DAO abstracts the *database*. a Repository abstracts the *data*.

Table 3 — Repository characteristics

Property	What it means
Language	Domain models, entities, value objects
Boundary	One class per <i>entity</i> (Video, Channel, SearchQueue)
Return type	Pydantic models
Testability	Mock the Repository directly
Swap backend	Swap the adapter underneath

why both matter

in this refactor, `DatabaseAdapter` is the DAO. it provides `_execute`, `_fetch_one`, `_fetch_all`. the Repositories inherit from it and add domain methods. the consumer (tracker, hunter, scribe) only sees the Repository.

the layering is:



Post-Refactor: (What the repo looks like now)

there are five repositories, each in its own file under `atlas/repositories/`:

Table 4 — Repository layout

Repository	File	Entity Model
<code>VideoRepository</code>	<code>video.py</code>	<code>Video, VideoStats</code>
<code>ChannelRepository</code>	<code>channel.py</code>	<code>Channel, ChannelStats</code>
<code>SearchQueueRepository</code>	<code>search_queue.py</code>	<code>SearchQueueItem</code>
<code>WatchlistRepository</code>	<code>watchlist.py</code>	<code>WatchlistItem</code>
<code>EventRepository</code>	<code>event.py</code>	<code>SystemEvent</code>

each repo owns its table. no cross-table concerns unless they're genuinely coupled (e.g. `ingest_video_metadata` still upserts a channel skeleton, because that happens in a single transaction).

before vs after: the tracker

this is the best concrete example. here is what the tracker's `fetch_targets_task` looked like before:

```
# BEFORE — MaiaDAO
from atlas.adapters.maia import MaiaDAO

dao = MaiaDAO()
targets = await dao.fetch_tracker_targets(batch_size)
# targets is List[Dict] — you pray the keys are right
```

and now:

```
# AFTER — VideoRepository
from atlas.repositories import VideoRepository

video_repo = VideoRepository()
targets = await video_repo.fetch_tracker_targets(batch_size)
return [t.model_dump() for t in targets]
# targets is List[Video] — typed, validated, has .model_dump()
```

the difference is the return type. the old code returned raw dicts from SQL. the new code returns `Video` domain models — Pydantic objects with typed fields, validation, and serialization methods.

before vs after: the stats update

the old `update_stats` used `dao.update_video_stats_batch` and `dao.log_video_stats_batch`, which were two separate methods on `MaiaDAO` that each took raw dicts:

```
# BEFORE
dao = MaiaDAO()
await dao.update_video_stats_batch(items)
```

```
# AFTER
video_repo = VideoRepository()
stats_list = [
    VideoStats(
        video_id=item["id"],
        views=int(stats.get("viewCount", 0)),
        likes=int(stats.get("likeCount", 0)),
        timestamp=datetime.now(timezone.utc),
    )
    for item in items
]
await video_repo.log_stats_batch(stats_list)
```

yes, it is more lines. but every field is named and typed. the old version was

`dao.log_video_stats_batch(items)` where `items` was a blob of YouTube API JSON. you had to read the DAO to know what keys it expected.

before vs after: the unit tests

old tests patched `MaiaDAO` at the module level. one wrong import path and the patch silently did nothing:

```
# BEFORE — fragile patch path
@patch("maia.tracker.flow.MaiaDAO")
def test_tracker(mock_dao):
    mock_dao.return_value.fetch_tracker_targets = AsyncMock(...)
```

new tests patch individual repositories. the patch target matches the actual import:

```
# AFTER — patches match the real import
@patch("maia.tracker.flow.VideoRepository")
def test_tracker(mock_video_repo):
    mock_video_repo.return_value.fetch_tracker_targets = AsyncMock(...)
```

and integration tests that used `MaiaDAO()` now use `VideoRepository()`:

```
# BEFORE
@pytest_asyncio.fixture
async def dao(fresh_db):
    from atlas.adapters.maia import MaiaDAO
    yield MaiaDAO() # raises ImportError after removal

# AFTER
@pytest_asyncio.fixture
async def video_repo(fresh_db):
    from atlas.repositories import VideoRepository
    yield VideoRepository()
```

Why These Patterns Fit This Problem

I chose Repository over DAO here for reasons specific to this codebase. not every project needs this split.

the pipeline has distinct entity lifecycles

the maia pipeline moves a video through stages: discovered → queue → ingested → transcribed → visualized → tracked → archived.

each stage touches different tables. hunter touches `search_queue`, `videos`, `channels`. tracker touches `videos`, `video_stats_log`. scribe touches `videos`, vault. janitor touches `video_stats_log`, vault, `videos`.

grouping all of these into one class meant the DAO had to know about every stage of every pipeline. that is not cohesion — it is coupling.

Table 5 — Which entities each agent touches

Agent	Tables read/written	Repositories it uses now
Hunter	<code>search_queue</code> , <code>videos</code> , <code>channels</code>	<code>SearchQueueRepository</code> , <code>VideoRepository</code> , <code>ChannelRepository</code>
Tracker	<code>videos</code> , <code>video_stats_log</code>	<code>VideoRepository</code>
Scribe	<code>videos</code> , vault	<code>VideoRepository</code>
Painter	<code>videos</code> , vault	<code>VideoRepository</code>
Janitor	<code>video_stats_log</code> , vault, <code>videos</code>	<code>VideoRepository</code>
Archeologist	<code>videos</code> , <code>search_queue</code>	<code>VideoRepository</code> , <code>SearchQueueRepository</code>

each agent now imports only the repositories it needs. the dependency graph is explicit.

the async + Prefect architecture rewards stateless objects

repositories are stateless. you instantiate one per task, use it, and drop it. this works naturally with Prefect's task model:

```
@task
async def fetch_targets_task(batch_size: int):
    repo = VideoRepository() # created and thrown away per task
    return await repo.fetch_tracker_targets(batch_size)
```

you do not need a shared DAO instance. you do not need a connection pool passed through every task. the default constructor uses the global pool, but you can inject a different one per task if you need to.

vault is not a database backend

the old code treated vault (HF/GCP) as a "backend" for the DAO, with methods like `archive_cold_stats` that wrote to both Postgres and vault in one method. this was the wrong abstraction.

vault is a cold-tier object store. it stores frames, transcripts, and parquet archives. Postgres stores the hot index. they serve different purposes and should be in different classes.

in the new design, `VideoRepository.archive_cold_stats` still touches vault (that's a PHASE-4 TODO to extract to a service), but the boundary is clear: the repository handles SQL, and vault is accessed through `atlas.vault.get_vault()`. they are separate concerns that happen to be called from the same workflow method.

Features Unlocked By This Refactor

swappable backends

`DatabaseAdapter` currently imports the global `db` singleton at module level. a follow-up change (already discussed) would make the pool injectable via the constructor:

```
class DatabaseAdapter:
    def __init__(self, db_pool=None):
        if db_pool is None:
            from atlas.db import db
            db_pool = db
        self._db = db_pool
```

this would let us:

- use a mock pool in unit tests (no Postgres needed at all)
- point `VideoRepository` at a read replica for analytics queries
- use SQLite for local development without changing any repo code
- spin up an ephemeral test database per test function

the repos themselves do not need to change. they inherit the adapter. the swap happens in one place — the constructor call.

better MLOps extensibility

because each repository is a concrete class with a well-defined interface, you can wrap them:

```
class LoggedVideoRepository(VideoRepository):
    async def fetch_tracker_targets(self, batch_size):
        start = time.time()
        result = await super().fetch_tracker_targets(batch_size)
        logger.info(f"fetch_tracker_targets({batch_size}) took {time.time() - start:.2f}s")
        return result
```

this pattern works for:

- **metrics** — wrap repos with Prometheus counters per method
- **caching** — `CachedVideoRepository` checks Redis before hitting Postgres
- **audit logging** — `AuditedEventRepository` logs every event dispatch
- **circuit breakers** — `ResilientVideoRepository` fails fast when Postgres is down
- **MLflow tracking** — log query shapes and response sizes to MLflow for cost analysis

none of these require changing the repos themselves. they are decorators in the classic sense, enabled by the fact that each repo is a single class with a focused interface.

granular testing

before this refactor, testing the tracker's stats update logic required mocking **MaiaDAO** — which had 30+ methods. you either mocked all of them (brittle) or used a real DAO (slow).

now you mock **VideoRepository** — which has methods only about videos. the mock surface is smaller and the intent is clearer:

```
@patch("maia.tracker.flow.VideoRepository")
def test_update_stats_handles_api_error(mock_repo):
    mock_repo.return_value.fetch_tracker_targets = AsyncMock(return_value=[...])
    mock_repo.return_value.update_stats_batch = AsyncMock()
    mock_repo.return_value.log_stats_batch = AsyncMock()

    result = await update_stats_task(videos, executor)
    assert result == 0 # graceful failure
```

you can read this test and know exactly what it's testing. you do not need to understand the entire DAO to understand one test.

independent schema evolution

videos table schema can change without touching **channels** or **search_queue**. the **VideoRepository** owns its migration. the **ChannelRepository** owns its migration.

this sounds trivial, but with a monolithic DAO, a schema change meant editing the one class that touched every table. you were one bad merge away from breaking the entire pipeline.

now each repo is a separate file with separate tests. you can refactor **VideoRepository** without worrying about whether **ChannelRepository** still works.

What I Would Do Differently Next Time

honestly, I would not build **MaiaDAO** at all. I would start with per-entity repositories from day one.

the monolithic DAO felt like the right choice at the start because the project was small and "one class for all database access" is the simplest possible design. but it did not scale past about 15 methods. the inflection point was when I added vault operations inside the DAO — that is when it became clear the abstraction was wrong.

a better rule of thumb is: if a class has methods that touch two different storage systems, split it. if a class has methods that serve two different agents, split it. if a class has more than 20 methods, split it.

the repository pattern is not fancy. it is just "one class per entity." the entities in this system are **Video**, **Channel**, **SearchQueueItem**, **WatchlistItem**, and **SystemEvent**. five classes. each with a single responsibility. the code is easier to read, easier to test, and easier to change.